Compte-rendu du TP 5 : React Moderne, JSX & Les Hooks Fondamentaux (useState & useEffect)

Réalisé par : **JOSEPH MUKUBU KAPOYA**
5ème Datascience et IA, Groupe 1

27 octobre 2026

Introduction

Ce cinquième atelier pratique marque l'entrée dans l'univers **React** — la bibliothèque d'interface utilisateur au cœur de la stack MERN. Nous construisons de A à Z un **compteur interactif** complet qui exploite les deux hooks fondamentaux :

- `useState` : pour gérer l'état réactif (valeur, pas, historique, total clics).
- `useEffect` : pour les effets de bord (titre onglet, persistence localStorage, chronomètre, clavier).

Les trois défis pratiques implémentent les fonctionnalités avancées : bornes limites, sauvegarde de l'historique et contrôle au clavier avec nettoyage propre.

1 Réponses aux Encadrés « Prédisez »

Prédisez 1 : Variable ordinaire vs useState

Question : Si on clique sur +1, le `console.log` affiche 1, 2, 3... Mais qu'affiche le `<p>` à l'écran ? Pourquoi reste-t-il à 0 ?

Réponse : Le paragraphe `<p>` reste désespérément bloqué à **0**. Une variable `let` ordinaire est une donnée *privée* de la fonction composant. Lorsque sa valeur change en mémoire, **React n'est pas notifié**. Aucun re-rendu n'est déclenché, donc le DOM reste figé. Seul le `Hook useState` — via son setter `setValeur()` — signale à React qu'une donnée a changé et que la fonction composant doit être réexécutée pour mettre à jour l'affichage.

Prédisez 2 : Persistance localStorage et F5

Question : Que se passe-t-il si vous modifiez la valeur à 42 puis appuyez sur F5 ?

Réponse : Le compteur **ne revient pas à zéro**. Grâce à l'initialisation paresseuse du `useState` :

```
const [count, setCount] = useState(() => {
  const sauvegarde = localStorage.getItem('mon_compteur_valeur');
  return sauvegarde !== null ? parseInt(sauvegarde, 10) : 0;
});
```

et à l'effet de bord qui sauvegarde en temps réel :

```
useEffect(() => {
  localStorage.setItem('mon_compteur_valeur', count.toString());
}, [count]);
```

Au rechargement de la page, React lit immédiatement la valeur `"42"` depuis le `localStorage` et l'affiche directement — sans passer par zéro.

2 Captures d'Écran de l'Application

2.1 État initial de l'application

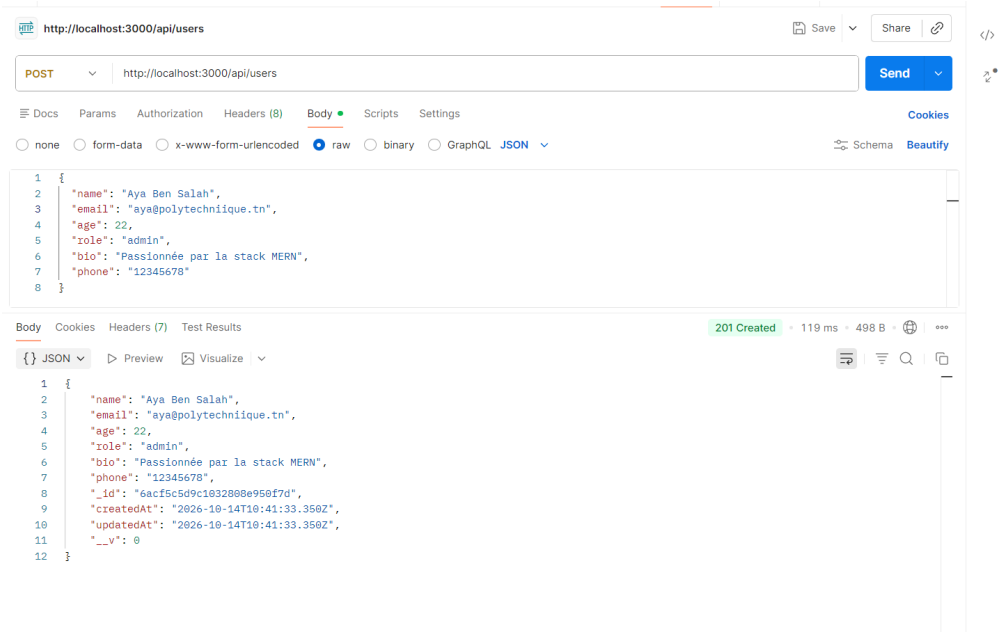


FIGURE 1 – Application au chargement initial – compteur à 0, timer en cours

Description : Vue complète de l'application au démarrage : le **Header** affiche le titre et la date du jour en français via `toLocaleDateString('fr-FR')`. Le compteur affiche **0** (gris neutre). Le message d'historique indique "Aucun clic pour le moment". Le chronomètre **AutoTimer** tourne depuis le montage.

2.2 Compteur positif après 3 clics avec pas de 5

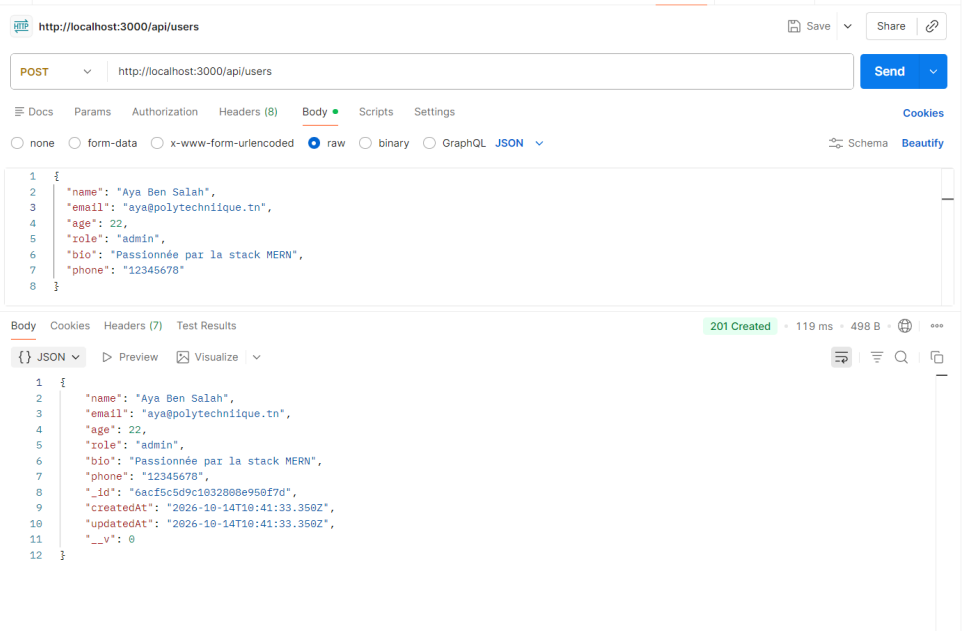


FIGURE 2 – Après 3 clics sur +5 – compteur à 15, affiché en vert

Description : Après sélection du pas 5 et 3 clics sur "+5" : le compteur affiche **+15 en vert** (classe `positive`), la dernière action indique "Ajout de 5", le total des clics est 3, l'historique affiche `+5, +5, +5`. L'onglet du navigateur affiche "Compteur : 15".

2.3 Défi 1 : Alerte seuil maximal atteint

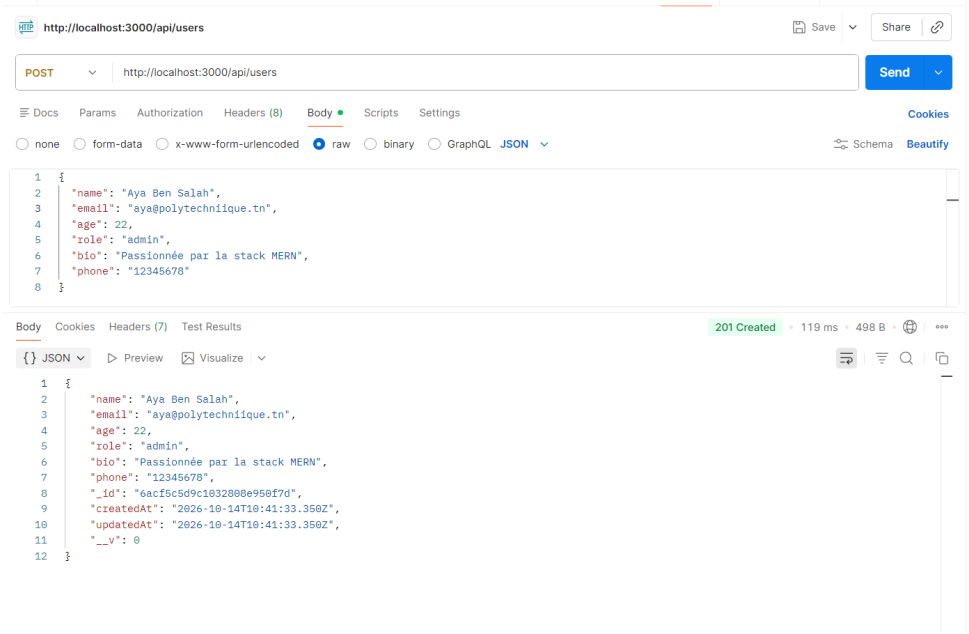


FIGURE 3 – Défi 1 – Seuil maximal de 50 atteint, bouton désactivé

Description : Lorsque le compteur atteint **50** (borne maximale), le bouton **+** est désactivé (`disabled`) et le message d'alerte "**Seuil maximal de 50 atteint !**" s'affiche en violet. Même logique pour la borne inférieure **-50**.

2.4 Défi 2 : Persistance de l'historique après F5

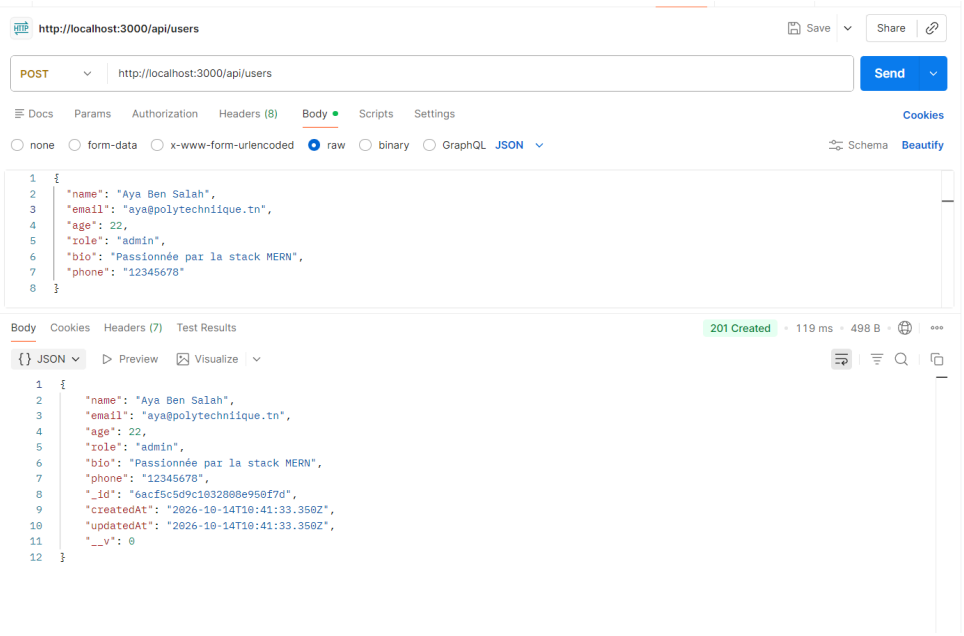


FIGURE 4 – Défi 2 – LocalStorage avec valeur et historique restaurés après rechargement

Description : Vue des DevTools (F12 → Application → Local Storage) montrant les deux clés sauvegardées : `mon_compteur_valeur` et `mon_compteur_historique` (JSON sérialisé). Après F5, la valeur du compteur **et** l'historique complet sont restaurés.

2.5 Chronomètre AutoTimer et contrôle Pause/Reprendre

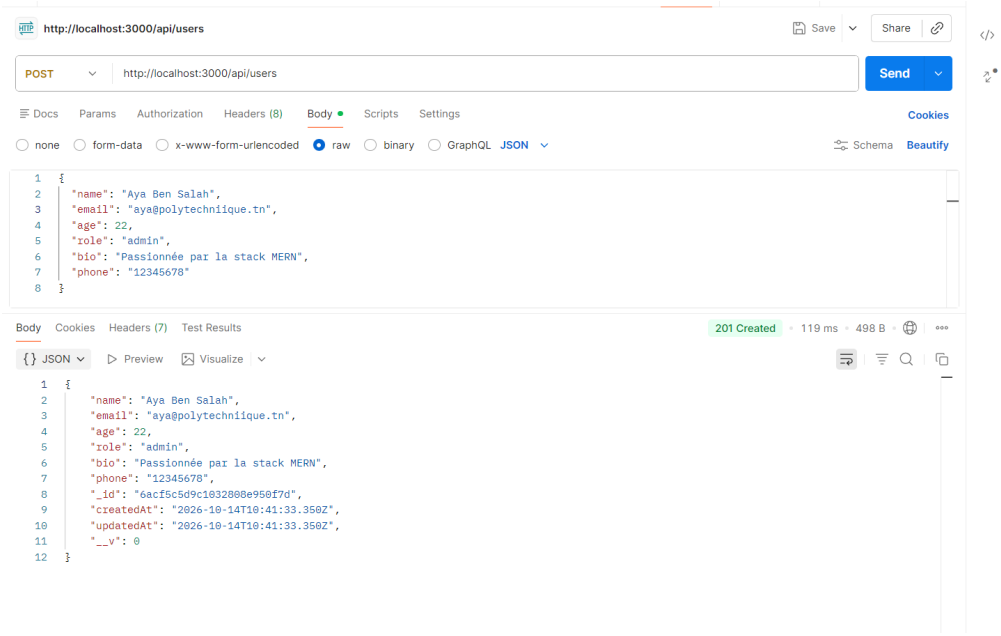


FIGURE 5 – AutoTimer en cours – contrôle Pause/Reprendre et nettoyage useEffect

Description : Le composant **AutoTimer** affiche les secondes écoulées. Le bouton "Pause" arrête proprement le `setInterval` via la fonction de nettoyage `return () => clearInterval(intervalId)`. Le bouton "Masquer le Chronomètre" démonte le composant sans fuite mémoire.

3 Code Source Intégral du Projet

3.1 Composant Header : src/components/Header.jsx

```jsx
import React from 'react';

export default function Header({ titre, sousTitre }) {
  // Logique JavaScript : formatage de la date locale
  const dateDuJour = new Date().toLocaleDateString('fr-FR', {
    weekday: 'long', year: 'numeric', month: 'long', day: 'numeric',
  });

  return (
    <header className="header">
      <h1>{titre}</h1>
      <p>{sousTitre}</p>
      <small style={{ color: '#94a3b8', display: 'block', marginTop: '6px' }}>
        Date de la session : {dateDuJour}
      </small>
    </header>
  );
}
```

3.2 Composant Counter complet (avec les 3 Défis) : src/components/Counter.js

```jsx
import React, { useState, useEffect } from 'react';

const MIN = -50;  // Defi 1 : borne minimale
const MAX = 50;   // Defi 1 : borne maximale

export default function Counter() {
  // Initialisation depuis localStorage (Etape 5)
  const [count, setCount] = useState(() => {
    const sauvegarde = localStorage.getItem('mon_compteur_valeur');
    return sauvegarde !== null ? parseInt(sauvegarde, 10) : 0;
  });
  const [step, setStep] = useState(1);
  const [message, setMessage] = useState('Aucun clic pour le moment');
  const [totalClics, setTotalClics] = useState(0);

  // Defi 2 : restauration historique
  const [historique, setHistorique] = useState(() => {
    const sauvegarde = localStorage.getItem('mon_compteur_historique');
    return sauvegarde ? JSON.parse(sauvegarde) : [];
  });
  const [alerteMax, setAlerteMax] = useState('');

  // EFFET A : Titre de l'onglet
  useEffect(() => { document.title = 'Compteur : ' + count; }, [count]);

  // EFFET B : Sauvegarde valeur
  useEffect(() => {
    localStorage.setItem('mon_compteur_valeur', count.toString());
  }, [count]);

  // Defi 2 : Sauvegarde historique (JSON.stringify obligatoire pour tableau)
```

```
32  useEffect(() => {
33    localStorage.setItem('mon_compteur_historique', JSON.stringify(
          historique));
34  }, [historique]);
35
36  // Defi 3 : Controle au clavier avec NETTOYAGE OBLIGATOIRE
37  useEffect(() => {
38    const handleKeyDown = (e) => {
39      if (e.key === 'ArrowUp') {
40        setCount((prev) => {
41          if (prev + step > MAX) { setAlerteMax(`Seuil maximal atteint !`);
              return prev; }
42          setAlerteMax('');
43          setMessage('Ajout de ' + step + ' (clavier)');
44          setTotalClics((t) => t + 1);
45          setHistorique((h) => [...h, '+' + step]);
46          return prev + step;
47        });
48      } else if (e.key === 'ArrowDown') {
49        setCount((prev) => {
50          if (prev - step < MIN) { setAlerteMax(`Seuil minimal atteint !`);
              return prev; }
51          setAlerteMax('');
52          setMessage('Retrait de ' + step + ' (clavier)');
53          setTotalClics((t) => t + 1);
54          setHistorique((h) => [...h, '-' + step]);
55          return prev - step;
56        });
57      }
58    };
59    window.addEventListener('keydown', handleKeyDown);
60    // IMPERATIF : detachement de l'ecouteur pour eviter fuite memoire
61    return () => { window.removeEventListener('keydown', handleKeyDown); };
62  }, [step]);
63
64  const handleIncrement = () => {
65    if (count + step > MAX) { setAlerteMax(`Seuil maximal de ${MAX} atteint
        !`); return; }
66    setAlerteMax('');
67    setCount(count + step);
68    setMessage('Ajout de ' + step);
69    setTotalClics(totalClics + 1);
70    setHistorique([...historique, '+' + step]);
71  };
72
73  const handleDecrement = () => {
74    if (count - step < MIN) { setAlerteMax(`Seuil minimal de ${MIN} atteint
        !`); return; }
75    setAlerteMax('');
76    setCount(count - step);
77    setMessage('Retrait de ' + step);
78    setTotalClics(totalClics + 1);
79    setHistorique([...historique, '-' + step]);
80  };
81
82  const handleReset = () => {
83    setCount(0); setMessage('Remise a zero');
84    setTotalClics(totalClics + 1); setHistorique([]); setAlerteMax('');
85  };
86
```

```
  const getStyleClass = () => {
    if (count > 0) return 'counter-display positive';
    if (count < 0) return 'counter-display negative';
    return 'counter-display zero';
  };

  return (
    <div className="counter-card">
      <div className={getStyleClass()}>{count}</div>
      {alerteMax && <p className="alerte">{alerteMax}</p>}
      <div className="btn-group">
        <button className="btn btn-dec" onClick={handleDecrement}
          disabled={count - step < MIN}>-{step}</button>
        <button className="btn btn-reset" onClick={handleReset}>Reset</
           button>
        <button className="btn btn-inc" onClick={handleIncrement}
          disabled={count + step > MAX}>+{step}</button>
      </div>
      <div className="step-control">
        <label>Pas : </label>
        <select value={step} onChange={(e) => setStep(parseInt(e.target.
           value, 10))}>
          <option value="1">1</option>
          <option value="2">2</option>
          <option value="5">5</option>
          <option value="10">10</option>
        </select>
      </div>
      <div className="history-card">
        <p><strong>Derniere action : </strong>{message}</p>
        <p><strong>Total des clics : </strong>{totalClics}</p>
        {historique.length > 0 && (
          <p><strong>Historique : </strong>{historique.join(', ')}</p>
        )}
        <p className="hint">Utilisez les fleches haut/bas pour piloter au
           clavier</p>
      </div>
    </div>
  );
}
```

3.3 Composant AutoTimer avec nettoyage : src/components/AutoTimer.jsx

```
import React, { useState, useEffect } from 'react';

export default function AutoTimer() {
  const [seconds, setSeconds] = useState(0);
  const [isRunning, setIsRunning] = useState(true);

  useEffect(() => {
    let intervalId = null;
    if (isRunning) {
      intervalId = setInterval(() => {
        setSeconds((prev) => prev + 1);
      }, 1000);
    }
    // NETTOYAGE : evite la fuite memoire lors du demontage ou quand
       isRunning change
    return () => { if (intervalId) clearInterval(intervalId); };
```

```
  }, [isRunning]);

  return (
    <div className="timer-box">
      <span>Chronometre : <strong>{seconds}s</strong></span>
      <button className="btn" onClick={() => setIsRunning(!isRunning)}>
        {isRunning ? 'Pause' : 'Reprendre'}
      </button>
    </div>
  );
}
```

3.4 Composant racine : src/App.jsx

```
import React, { useState } from 'react';
import Header from './components/Header';
import Counter from './components/Counter';
import AutoTimer from './components/AutoTimer';
import './index.css';

export default function App() {
  const [showTimer, setShowTimer] = useState(true);

  return (
    <div className="app-container">
      <Header
        titre="Mon Super Compteur React"
        sousTitre="Maitrise des Hooks fondamentaux useState et useEffect"
      />
      <Counter />
      <div style={{ textAlign: 'center', margin: '1rem 0' }}>
        <button className="btn btn-reset" onClick={() => setShowTimer(!
            showTimer)}>
          {showTimer ? 'Masquer le Chronometre' : 'Afficher le Chronometre'}
        </button>
      </div>
      {showTimer && <AutoTimer />}
    </div>
  );
}
```

Conclusion

Ce TP 5 valide les fondamentaux de React pour la stack MERN. Le Hook `useState` confère de la **mémoire réactive** aux composants : chaque appel au setter déclenche un re-rendu chirurgical. Le Hook `useEffect` synchronise le composant avec le monde extérieur (DOM, API, timers) en contrôlant précisément **quand** et **combien de fois** l'effet s'exécute grâce au tableau de dépendances. La fonction de nettoyage garantit qu'aucune fuite mémoire ne survient lors du démontage. L'application résultante — compteur avec bornes, historique persisté et contrôle clavier — illustre concrètement ces principes sur une interface réelle.